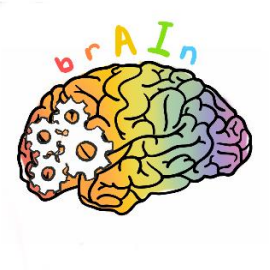
302 Python Project
brAIn



Team 44

[redacted]

Abstract

Handwritten Digit Recognition has been part of machine learning for over 30 years and it has helped the modern world progress and set a steppingstone in helping machine learning progress and evolve.

By using Python and its vast array of modules, brAIn was able to develop a program which is able to take input from a user and in order to recognize said input using an AI model.

We made use of the MNIST database as a baseline for our models classifications as it has a large library of 70,000 images.

We developed four models for the user to test digits with which has variable recognition accuracies which helps provide a sense to how machine learning can be affected dependent on what models that one can acquire or produce.

1. Introduction

brAIn is an AI company that has been working on machine learning of AI models and we have focused on handwritten digit recognition. Our digit analyzer is aimed to be at 98% accuracy in our recommended AI model.

2. Planning

2.1.Literature Review

2.1.1. Handwritten Digit Recognition Using PyTorch — Intro To Neural Networks [1]

This is a relatively simple approach to building a model for handwritten digit recognition. They start with first of all importing a database for images to train and test the model with, in this case they have chose the MNIST database. This is in our opinion the best database to use as it provides a clean database with a set image type where it is size-normalized and centered on 28 by 28 pixel image, which also offers a variety of varying shuffled images and a large number of them – 70,000 to be exact.

We also used the PyTorch module as it offered the easiest implementation to download and manipulate our datasets into values that could be understood by the system. Also offered more function to allow us to input our own digit into our model and to interact with

our model to output predictions from tensor weights that the system would be giving out.

The ReLu activation function was also quite useful in the sense that we were able to pass through only positive values while negatives values are ignored or else our model outputs and predictions would be harder to have been understood due to the negatives having an impact on the tensor weights.

2.1.2. MNIST Handwritten Digit Recognition in Keras, Epoch, Iterations, Batch Size [2 - 4]

Epoch number and batch size play and important role in developing a neural network.

Epoch number dictates how many iterations our model wants to go through in order to improve accuracy, a low number of epoch will underfit a model, a high number of epochs will cause the model to be overfitted whilst there is no right number of epochs in order to fit a model a right number of epochs can be found through trial and error. By testing our model between 5 and 20 epochs we decided that 15 epochs was the model type that gave us the best accuracies by fitting to the data that we have.

Batch size affects our neural network as we can pick how many samples we want to train at once through the given epochs. While a low batch size will produce result in rapid learning but it provides a higher variance in classification accuracy – the time it takes to learn will be longer but the model will have varying classification data as it is overfitting the data to match every sample.

Larger batch sizes slows down the learning process but produces lower variance in classification accuracy – time for model to learn will be much shorter with classification data underfitting the model hence over some digits the classification can be the same.

2.2.Project Setup

2.2.1. Environment

A python environment (Python 3.8) was chosen as it is a high level programming language that offers multiple modules which enabled us to achieve a much more elegant solution, e.g. PyTorch, PyQt5, TensorFlow.

Python has a wide range of open source frameworks and development tools to suit the needs of the program that we want to build.

2.2.2. Tools

PyTorch [site]

An open source machine learning library which is used for applications such as computer vision and natural language processing which helped us in training our AI model's Deep Neural Network (DNN).

PyQt5 [site]

A set of cross-platform C++ libraries that implement high-level APIs for accessing many aspects of modern desktop and mobile systems, which also has a hand in traditional UI development, which was used primarily in developing our GUI for the program

torchvision [site]

The torchvision package consists of popular datasets, model architectures, and common image transformations for computer vision. Used to access databases such as MNIST and allowed us to interact with the data.

NumPy [site]

NumPy is the fundamental package for scientific computing in Python. It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices). For our model to be able to output probabilities from tensors this tool was quite useful.

Pillow [site]

The Python Imaging Library adds image processing capabilities to your Python interpreter. This library provides extensive file format support, an efficient internal representation, and fairly powerful image processing capabilities. This library helped with our image preprocessing to be inputted into our medal.

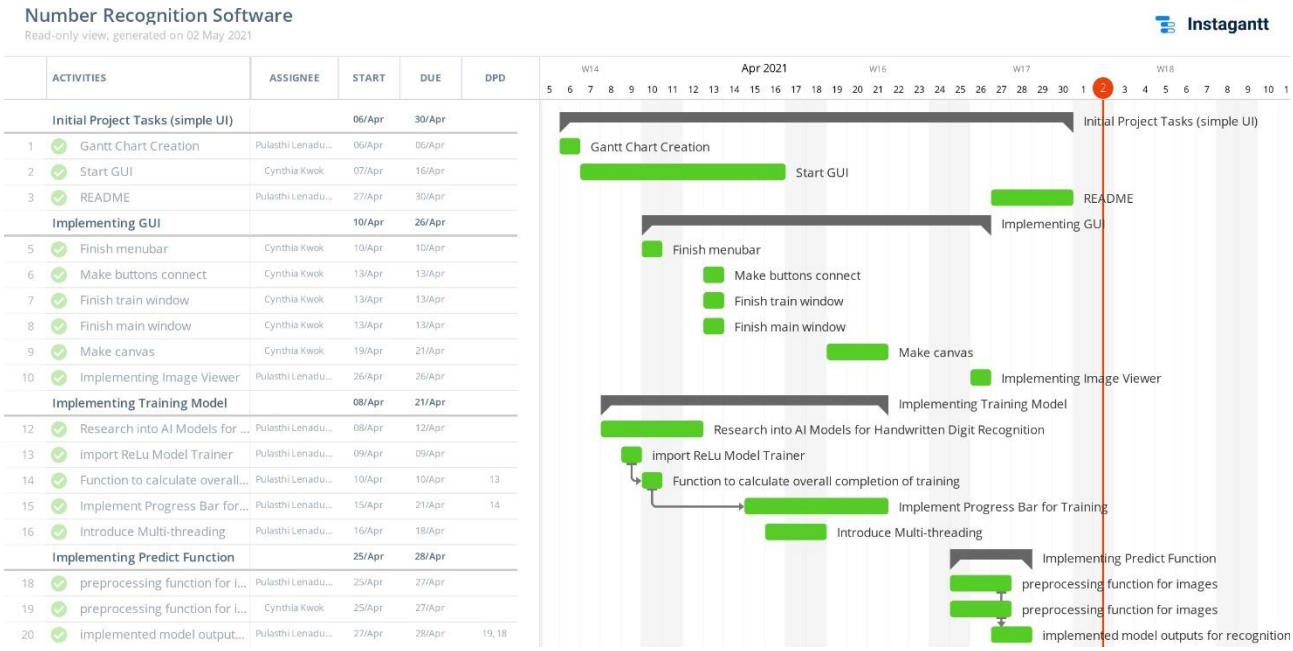


Figure 1: Gantt Chart for Number Recognition Software

Matplotlib [site]

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. In order to plot our probabilities onto a graph, this module proved to be quite useful

2.3.Gantt Chart

Roles and schedules done using Asana Fig. 1.

3. Design and Software Architecture

3.1. Purpose of System

The goal of this project is to build a program which is able to recognize hand written digits.

This branch of AI can be said to be quite old as it can be seen as early as 1989 where postal services used a machine-enabled ZIP code parser. Soon afterwards financial institutions used this branch of AI in the same fashion where they developed automatic parsers for bank check processing. There are other applications such as form data entry as well.

Having an AI in charge of very important applications such as these makes it much more efficient and no room for human error will ever have to arise. Even if the AI programs used for these may experience bugs and such, it is always a quick fix as bugs can be traced and solved; the same could not be said for human error as sometimes it can be hard to track where someone went wrong, humans thinking process and application process does tend to vary over time whereas an AI will be set on a single process.

We want to build a program which would help in these applications as well as data processing, vehicle license-plate recognition, historical document preservation, old document automation in libraries, etc. Our society is in the transition phase from analog to digital so brAIn believes that in helping developing a program such as handwritten digit recognition we can help society fully move much more smoothly and efficiently into an era where it is fully digital.

3.2.Database, Model and Methods

The MNIST database of handwritten digits offers a training set of 60,000 examples and a test set of 10,000 examples. It is a subset of a larger set available from NIST. The digits have been size-normalized and centered in a fixed-size image [5].

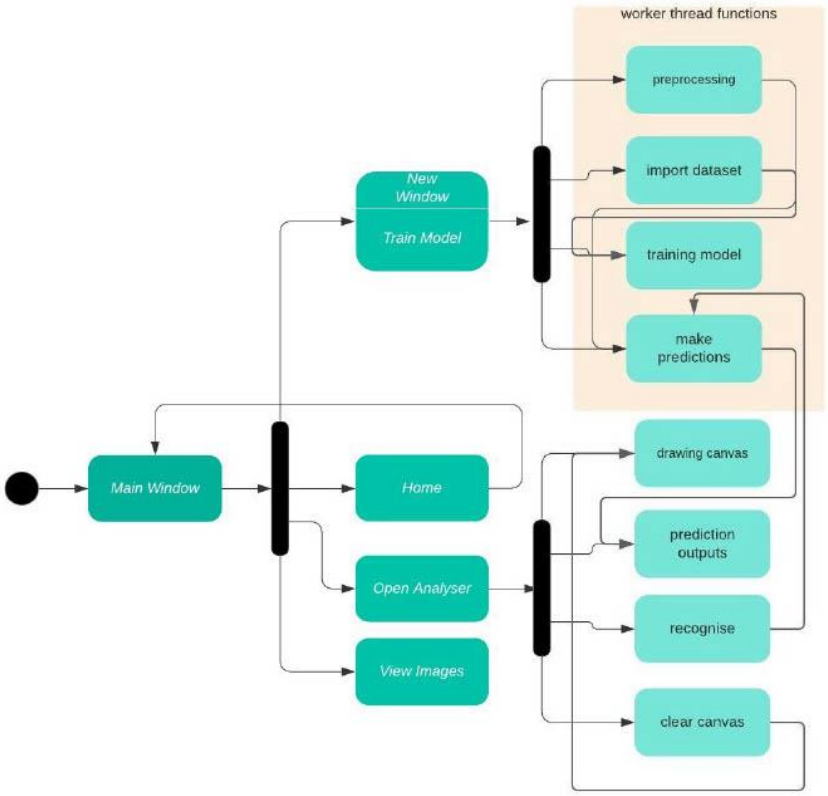


Figure 2: System Diagram

No need to make major modifications to it as it has been optimized for use in pattern recognition methods without the need for preprocessing and formatting.

We opted for the use of the MNIST database as it was the database which had a very consistent data type, the images are size-normalized and centered in a 28 by 28 image. This gave us a simple and easy-to-achieve criteria for our own input images into the model.

The use of the two types of datasets helped us train our model using the training dataset of 60,000 examples and test the accuracy of our model using the testing dataset of 10,000 examples.

3.3.System Diagram

System Diagram using Lucid Chart Fig. 2.

3.4. Components

3.4.1. Main Window

Comprised of Home, View Images, Analyse. The user is able to navigate to Home, Train Model, Quit, Open Analyser, and View Testing Images from this window

The Home tab is the main page that opens when the program launches and describes to the user what the main functionality of the program is.

Open Analyser is where the user can provide input of a handwritten digit and prompt the AI model to recognize which digit it is that they wrote or clear the canvas. They can also see the predictions that AI model outputs on this window.

3.4.2. Train Model

In this module the user is able to pick a desired model from the models listed and train the AI model, when a model is picked another model will not be able to be picked until after the AI has finished training it's current model.

The accuracy of the model will be displayed as the model is being trained and will update as each of the epochs is trained and finally update to the overall accuracy of the model after it has finished training.

The progress bar will depict the total completion of training over time and will unlock the model buttons as it reaches 100%

3.4.3. ReLu trainer

This is where the AI model is primarily trained using the different model variations specified in *Train Model* we are able to train a model for a different number of epochs.

The MNIST dataset is also imported in this module to be used by our AI Model.

In order for our model to recognize digits, preprocessing is applied to the canvas image in *Open Analyser* to size-normalized, centered 28 by 28 pixel image. This image is then returned as a returned as a 1D tensor value in order to be put into our model.

The prediction outputs is calculated in this module by converting tensor weights into probabilities.

The overall accuracy and completion of training is returned in this module using getter functions which allow for a clean approach.

3.4.4. Worker Threads

This module is specifically designed to separate different threads in the thread pool.

We primarily used this module in order to initialize different worker threads for our trainer to function whilst the GUI is on the main thread.

3.5.Benefit of Architecture

There were a few issues we faced as we progressed through the project such as our GUI freezing and tediousness to use the application.

To resolve our GUI freezing we used multithreading and initialized our GUI in the main thread and our AI model in a worker thread. This helped us train our model and interact with the GUI at the same time.

The application was getting tedious to use as we had to keep switching between tabs to interact with the different components such as training, analyzing and viewing images. Hence we resolved it by having training in a separate window as it did not require the same thread to operate in.

In order for our program to be easily understood and edited by other sources who would be new to our code, we made some of the larger modules such as the Canvas, AI Model, Training Window, Main Window, Worker Threads in different modules and this way it was easy to navigate to which module you would want to view or edit and promoted modularity.

4. AI Model

Our AI model makes use of a Multi- Layer Perceptron (MLP) in order to build our neural network. MLP is a type of artificial neural network (ANN).

Simplest MLP consists of at least three layers of nodes: an input layer, a hidden layer and an output layer.

We designed our training model with 5 hidden layers and with the input and output layers we have 7 layers. The reason for adding 5 hidden layers is because we wanted to build an AI model with a Deep Neural Network (DNN) instead of an Artificial Neural Network (ANN).

For our DNN we used the ReLu activation function as it only passes through positive values which helps for faster convergence and the model will not suffer from the Vanishing Gradient problem [6].

brAIn adopted the MNIST database as it provided the model with the most appropriate dataset being a size-normalized, centered on 28 by 28 pixel image sets with a large variety of 70,000 images.

	0	1	2	3	4	5	6	7	8	9
0	9	0	0	0	0	0		0	1	0
1	0	2	1	1	0	0	0	0	6	0
2	0	0	10	0	0	0	0	0	0	0
3	0	0	0	10	0	0	0	0	0	0
4	0	0	0	0	9	0	0	0	1	0
5	0	0	0	3	0	7	0	0	0	0
6	0	0	0	0	0	3	7	0	0	0
7	0	0	0	2	0	0	0	8	0	0
8	0	0	0	0	0	3	0	0	7	0
9	0	0	0	0	1	0	0	1	0	8

Table 1: Confusion Matrix of Model 3

Batch size of 64 was chosen as it proved to be the optimal number after going through trial and error between 32, 64 and 128 batch sizes.

We allowed the user to be able to choose the number of epochs as they should have an option on which model they would like to develop and what accuracies they may desire.

5. Results

5.1. Details of Tool

For our results we primarily used the Recommended Model which would be Model 3.

Model 3 is comprised of 15 epochs, and batch size is 64 which we deemed to be the optimal for fitting the model as we required.

5.2.Statistical Analysis

Using values from Table 1

$$accuracy = \frac{True\ Positive}{total\ dataset}$$
$$= \frac{77}{100} = 77\%$$

$$precision = \frac{TP}{TP + FP}$$
$$average = precision(0:9)$$
$$precision\ rate = 0.83$$

$$recall = \frac{TP}{TP + FN}$$
$$average = recall(0:9)$$
$$recall\ rate = 0.77$$

$$f1 = 2 * \frac{precision * recall}{precision + recall}$$
$$= 0.4$$

5.3.Recognition Results

Refer to Table 1 for recognition results

5.4.Discussion

The f1 score for our model is lower than expected even though we achieved a high accuracy in training our model.

This may be due to the batch size of 64, perhaps a smaller batch size of 32 would help in achieving more accurate prediction results

6. Conclusion

By using Python and its vast array of modules, brAIn was able to develop a program which is able to take input from a user and in order to recognize said input using an AI model.

We developed four models for the user to test digits with which has variable recognition accuracies which helps provide a sense to how machine learning can be affected dependent on what models that one can acquire or produce.

Our recommended model only gave us an f1 score of 0.4 which is lower than our expectations. We think that there is room for improvement regarding accuracies of outputs to be made and perhaps improving our GUI so it includes more functionality.

7. Future Works

The application that we have produced is rather intermediate as it only allows you to recognize one digit at a time.

brAIn believes that if we are able to progress our knowledge and polish our skills further in order to develop and improve on our base application we will be able to recognize larger numbers and have a higher accuracy with our predictions.

We believe that implementing machine learning in number recognition would be a first step in the way forward to a fully digital era and modernization

8. Acknowledgements

brAIn would like to thank COMPSYS 302 teaching team for providing support and guidelines in developing this project, and for providing the AI Model we have implemented in our application as "*ReLu Trainer*".

9. References

[1] A. Bose, *"Handwritten Digit Recognition Using PyTorch — Intro To Neural Networks"*, towards data science, Feb 17, 2019. Accessed on: April 28, 2021. [Online]. Available: https://towardsdatascience.com/handwritten-digit-mnist-pytorch-977b5338e627

[2] G. Koehler, *"MNIST Handwritten Digit Recognition in Keras"*, Nextjournal, February 18, 2020. Accessed on: April 28, 2021. [Online]. Available: https://nextjournal.com/gkoehler/digit-recognition-with-keras

[3] E | M, *"Epoch, Iterations, Batch Size"*, medium, May 7, 2019. Accessed on: April 28, 2021. [Online]. Available: https://medium.com/@ewuramaminka/epoch-iterations-batch-size-11fbbd4f0771

[4] J. Brownlee, *"How to Control the Stability of Training Neural Networks With the Batch Size"*, Machine Learning Mastery, January 21, 2019. Accessed on: April 28, 2021. [Online]. Available: https://machinelearningmastery.com/how-to-control-the-speed-and-stability-of-training-neural-networks-with-gradient-descent-batch-size/

[5] Y. LeCun and C. Cortes, "*MNIST handwritten digit database*", 2010. Accessed on: April 28, 2021. [Online]. Available: http://yann.lecun.com/exdb/mnist/

[6] P. Sharma. *"PyTorch Activation Functions – ReLU, Leaky ReLU, Sigmoid, Tanh and Softmax"*, Machine Learning Knowledge, March 20, 2021. Accessed on April 29, 2021. [Online]. Available: https://machinelearningknowledge.ai/pytorch-activation-functions-relu-leaky-relu-sigmoid-tanh-and-softmax/#:~:text=ReLU%20activation%20functions%20are%20ideal,output%20layer%20for%20multiclass%20classification.